# Wilo-Heizungspumpe trifft MicroPython

PWM-Feedback mit C-Natmodul auslesen

make Magazin · Projekt-Doku

April 2026 `github.com/gerontec/wilo_pwm`

---

*Ein Raspberry Pi Pico W regelt eine Wilo iPWM-Heizungspumpe per MQTT — und zeigt, wie man MicroPythons Grenzen elegant mit einer 364-Byte großen C-Erweiterung überwindet.*

---

## 1 Das Problem: Eine intelligente Pumpe schweigt

Moderne Heizungsumwälzpumpen wie die Wilo iPWM-Serie sind alles andere als dumme Verbraucher. Sie melden ihren Betriebsstatus über ein PWM-Feedback-Signal zurück: Ein 75-Hz-Signal, dessen Tastverhältnis Aufschluss über Drehzahl und Zustand gibt.

| **Duty-Cycle** | **Bedeutung** |
|---|---|
| 5–75 % | Normalbetrieb (Flow-/Leistungsfeedback) |
| 80 % | Abnormaler Betrieb |
| 85–90 % | Temporär gestoppt / Warnung |
| 95 % | Permanenter Fehler |
| >97,5 % | Interface beschädigt / Ausfall |

Die Herausforderung: Dieses Signal zuverlässig auszulesen — und gleichzeitig die Pumpe per PWM zu steuern, Messwerte per MQTT zu publizieren und das Ganze über WLAN erreichbar zu halten.

## 2 Hardware: Pico W als Schaltzentrale

Der Raspberry Pi Pico W übernimmt alle Aufgaben gleichzeitig. GPIO 0 liefert das PWM-Steuersignal (0–64 000 Steps, 800 Hz), GPIO 5 liest das Feedback-Signal der Pumpe. Über WLAN verbindet sich der Pico mit einem MQTT-Broker und publiziert alle 5 Sekunden einen vollständigen JSON-Payload: Duty-Cycle, Frequenz, Pumpenstatus, alle GPIO-Zustände, ADC-Werte, Uptime.

Die MQTT-Topics erlauben auch Fernsteuerung: `heatp/pump` akzeptiert Befehle wie `on`, `off`, `reset` oder direkte PWM-Werte. Ein 15-Minuten-Boost-Zyklus sorgt dafür, dass die Pumpe nie zu lange auf Minimalleistung läuft.

## 3 PWM-Feedback: Mehr als Flankenzählen

Das Feedback-Signal zu messen klingt einfach — ist es aber nicht. Man braucht sowohl die HIGH- als auch die LOW-Dauer jedes Pulses, um den echten Duty-Cycle zu berechnen. Einfaches Flankenzählen reicht nicht.

MicroPython löst das elegant mit Hardware-IRQ (`hard=True` auf GPIO 5), der bei jeder Flanke die Systemzeit ausliest. Das funktioniert bei 75 Hz problemlos — aber die Grenzen werden bei höheren Frequenzen schnell spürbar: Garbage-Collection-Pausen von bis zu 50 ms können Flanken verschlucken, und oberhalb von ∼1–2 kHz wird die reine Python-IRQ-Lösung unzuverlässig.

## 4 Die Lösung: 364 Bytes C

Statt die komplette MicroPython-Firmware neu zu bauen, nutzt das Projekt MicroPythons **natmod-System**: C-Code, der zu einer `.mpy`-Datei kompiliert wird und wie ein normales Python-Modul importiert werden kann — auf jeder Standard-MicroPython-Firmware.

Der Trick liegt in zwei direkten Registerzugriffen ohne pico-sdk:

Listing 1: Direkter RP2040-Registerzugriff

```c
// GPIO-Zustand und us-Timer direkt lesen
#define SIO_GPIO_IN  (*(volatile uint32_t*)0xD0000004u)
#define TIMER_TIMELR (*(volatile uint32_t*)0x4005400Cu)

static mp_obj_t mp_irq_cb(mp_obj_t buf_obj) {
    mp_buffer_info_t buf;
    mp_get_buffer_raise(buf_obj, &buf, MP_BUFFER_RW);
    pwmfb_state_t *s = (pwmfb_state_t*)buf.buf;

    uint32_t now     = TIMER_TIMELR;          // C-Speed
    uint8_t  current = (SIO_GPIO_IN >> s->pin_num) & 1u;
    uint32_t diff    = now - s->last_chg;

    if (diff >= 100u && current != s->last_state) {
        if (current) s->low_us  = diff;
        else         s->high_us = diff;
        s->last_upd = s->last_chg = now;
        s->last_state = current;
    }
    return mp_const_none;
}
```

Der State liegt in einem Python-`bytearray` auf dem Heap — das natmod-Format erlaubt keine C-Statics (`.bss size: 0`).

Der Build-Prozess ist denkbar einfach:

Listing 2: Natmod kompilieren

```bash
# Einmalig: MicroPython-Repo klonen
git clone --depth=1 https://github.com/micropython/micropython

# C-Modul bauen (armv6m = Cortex-M0+ = RP2040)
cd pwmfeedback_c
make     # -> _pwmfeedback.mpy  (364 Bytes, 0 Bytes BSS)
```

Das `.mpy`-File wird per WebREPL auf den Pico hochgeladen. MicroPython-Updates erfordern keinen Rebuild — die `.mpy`-Datei bleibt unverändert auf dem Dateisystem.

## 5 Deploy in einem Befehl

Listing 3: deploy.sh

```
./deploy.sh           # alle Dateien per WebREPL + MQTT-Reset
./deploy.sh main.py   # nur eine Datei deployen
```

Das Deploy-Script lädt geänderte Dateien auf den Pico und schickt anschließend einen Reset-Befehl per MQTT. Nach ∼13 Sekunden erscheint der neue Firmware-String im nächsten MQTT-Payload — Versionskontrolle inklusive:

`v2.33-min-pwm-feedback | rp2 v1.29.0 | Raspberry Pi Pico W`

## 6 Ergebnis und Ausblick

| **Ansatz** | **Max. Frequenz** | **Firmware-Build?** |
|---|---|---|
| Reines Python IRQ | ∼1–2 kHz | nein |
| C-Natmod (diese Lösung) | ∼10 kHz | nein |
| Custom Firmware + Core 1 | ∼1–5 MHz | ja |
| Custom Firmware + PIO | ∼62 MHz | ja + Assembly |

Für eine 75-Hz-Heizungspumpe ist das akademisch. Aber der Weg dorthin zeigt, wie weit man mit einem 4-Euro-Microcontroller, offenem Standard-Toolchain und 364 Bytes C kommt.

Wer den nächsten Schritt gehen will: Der RP2040 hat zwei Kerne. Core 1 kann als reiner C-Polling-Loop laufen — vollständig unabhängig von MicroPython, ohne GC-Pausen, mit direktem Registerzugriff. Core 0 behält dabei volle MicroPython-Funktionalität. Die Firmware dafür ist bereits gebaut.

---

**Code & Deploy-Script:** https://github.com/gerontec/wilo_pwm